CS315 · ALGORITHMS ANALYSIS & DESIGN

What Repeats

An analysis of five midterms: 472, 462, 461, 451 and 442

Muharib

What's inside

01 The bottom line

02 The repetition table

03 Where are the marks?

04 Answers to repeated questions

05 Study plan

01 The bottom line

I compared five midterms. Four of them (472, 462, 461 and 451) share nearly the same structure: a hash table question, a true/false and MCQ question, two dynamic programming or greedy questions, and a B-tree question.

Repeated word for word	Where
LCS: X = ACBAED, Y = ABCADF	472 · 462 · 451
Matrix chain: A=4×5, B=5×8, C=8×3, D=3×7	472 · 451
Hash table: [4, 13, 49, 6, 40, 33], size 9	472 · 451
T/F: Huffman, greedy solves all DP, and the MERGESORT equation	472 · 462 · 451
MCQ: the algorithm that uses past results	472 · 462 · 451
B-tree of order 5 with nearly the same numbers	472 · 451

So

The LCS question came with the same two strings in three midterms, and its answer is `ACAD` or `ABAD` with length 4. Learn the method because the instructor may change the strings, but start your revision from these questions.

02 The repetition table

The number in a cell is the question's mark. A ✓ means it appeared but the mark is

unknown, or it appeared as an item inside a question.

Topic	472	462	461	451	442	Lecture
Hash table: chaining, linear probing or double hashing	✓	5	5	4	✓	3
True / False + MCQ	✓	5	9	5	4	1 – 6
Longest common subsequence	✓	5	✓	4	—	4+
Matrix chain multiplication	✓	—	6	4	—	4+
Greedy knapsack (fractional)	—	5	5	—	—	4+
B-tree: build, insert and delete	✓	5	—	6	—	4+
Counting operations of **arrayMax**	—	—	—	2	2	1
Huffman: T/F, a code word or the running time	✓	✓	✓	✓	—	4+
Heaps, BST, trees and graphs	—	—	—	—	✓	—

"4+" means a lecture after the third.

03 Where are the marks?

Midterm	From Lectures 1 to 3	From the later lectures
462	About 7.5 of 25	About 17.5
461	About 9 of 25	About 16
451	About 7 of 25	About 18

Watch out

In every midterm, Lectures 1 to 3 carry only about a third of the marks. The other two thirds come from dynamic programming (LCS and matrix chain), greedy (knapsack, Huffman and activity selection) and B-trees.

04 Answers to repeated questions

Question	Answer
$T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + T(\frac{\text{MERGE}}{2})$	**False**
The recursive activity-selection problem algorithm takes a time of O(n log(n))	**Other value**: O(n)
An algorithm which uses the past results and uses them to find new results	**Dynamic programming**
f(n) is ω(g(n)) if is asymptotically ... than g(n)	**Strictly greater**
Best-case time complexity of linear search	**O(1)**
B-tree of degree 3	At most 3 children and 2 keys

05 Study plan

1 From Lectures 1 to 3

- Hash tables with all three methods. They appeared in all five. Practise with the Practice 2 file.
- The **arrayMax** counting table: `7n - 2`, and `7n` with the Temp line.
- True/false statements from the slides' wording: the definition of an algorithm, insertion sort "about n^2", the dictionary operations, and the log file.

2 The heavy questions

- LCS: diagonal + 1 when the letters match, otherwise the larger of top and left.
- Matrix chain: the cost is `p·q·r`, and fill the table from short chains to long.
- Fractional knapsack: sort by value ÷ weight and take in that order.
- B-tree: split when a node exceeds its maximum number of keys, and the middle key moves up.

All of them are solved step by step in the 462, 461 and 451 solution files.

3 The last day

Solve the 462 and 451 midterms in full, each against a 90-minute timer.